

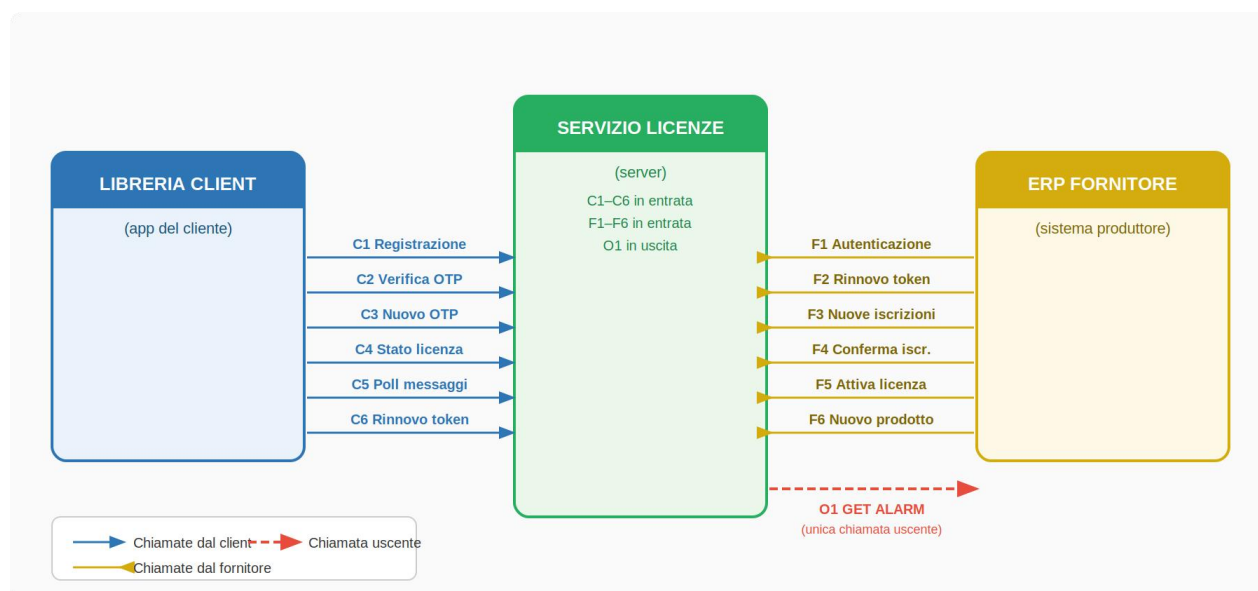
Analisi – Servizio Gestione Licenze

1. Panoramica generale

Il Servizio Gestione Licenze è un sistema che si posiziona tra la **libreria client** e il **sistema ERP del produttore**. Il servizio non inizia mai comunicazioni verso i clienti o il produttore di propria iniziativa, ad eccezione di un'unica notifica uscente verso l'ERP (il GET ALARM) e dei messaggi email/in-app schedulati.

Il suo scopo principale è **gestire le licenze software** dalla prima registrazione del cliente, all'attivazione della Trial Demo, fino alla Licenza Provvisoria, al rinnovo e alla scadenza. Ogni operazione transita attraverso questo servizio, che aggiorna costantemente lo stato delle licenze.

Architettura a tre livelli



Legenda:

- Frecce blu: chiamate libreria client (C1–C6);
- Frecce gialle: chiamate del produttore (F1–F8);
- Freccia rossa tratteggiata: GET ALARM (O1), unica chiamata uscente.

Punti chiave

- Il servizio è passivo: risponde alle chiamate, non le inizia (ad eccezione di O1 e messaggi schedulati)
- La P.IVA/VAT viene validata al momento della registrazione tramite il servizio VIES (sistema UE, copre tutti i paesi UE)
- Ogni cliente è identificato da una license key univoca generata con salt random al momento dell'attivazione

- Ad ogni attivazione o rinnovo viene generato un file crittografato con chiave di decrittazione per il funzionamento offline
- La frequenza delle chiamate C4 e C5 è configurabile dal produttore tramite `check_frequency_hours`
- I template dei messaggi sono gestiti in DB nella tabella `email_templates` con `event_code` e placeholder sostituibili
- Sono supportati quattro tipi di licenza: trial, monthly, annual, provisional
- Il sistema è multilingua (italiano e inglese) e predisposto per futura gestione multi-produttore

2. Descrizione dettagliata del funzionamento

Di seguito viene descritto il funzionamento completo del servizio seguendo l'ordine naturale dei processi: dalla configurazione iniziale del produttore, alla registrazione del cliente, fino agli scenari di scadenza, Licenza Provvisoria e casi eccezionali.

2.1 Configurazione iniziale del produttore

Prima che qualsiasi cliente possa registrarsi, il produttore deve autenticarsi sul servizio e registrare i propri prodotti. Questa fase avviene per ogni nuovo prodotto messo in commercio.

F1 – Autenticazione produttore

POST /api/vendor/auth/login

Il sistema ERP del produttore si autentica inviando la propria API key statica. Il servizio verifica la chiave nella tabella **vendors** e restituisce un JWT di breve durata e un refresh token valido un'ora.

Tabella coinvolta: vendors

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del produttore
name	VARCHAR(255)	Nome del produttore
api_key	VARCHAR(255)	Chiave API statica usata per l'autenticazione (hash)
erp_alarm_url	VARCHAR(500)	URL dell'endpoint GET ALARM dell'ERP del produttore
check_frequency_hours	INT	Frequenza in ore con cui la libreria client deve chiamare C4 e C5 — configurata dal produttore
created_at	TIMESTAMP	Data e ora di inserimento del record

Tabella coinvolta: vendor_tokens

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del record
vendor_id	INT	Riferimento al produttore
refresh_token	VARCHAR(512)	Refresh token (hash) — ruotato ad ogni utilizzo
expires_at	TIMESTAMP	Scadenza del refresh token (1 ora)
revoked	BOOL	TRUE se revocato anticipatamente
created_at	TIMESTAMP	Data e ora di emissione

F2 – Rinnovo token produttore

POST /api/vendor/token/refresh

Quando il JWT scade, il produttore usa il refresh token per ottenerne uno nuovo. Il refresh token viene ruotato ad ogni utilizzo (token rotation).

F6 – Registrazione nuovo prodotto

POST /api/vendor/products

Il produttore registra un nuovo prodotto comunicando la sua `product_key` e il nome. La chiave viene inserita nella tabella **products** e da quel momento è valida per le registrazioni dei clienti che acquisteranno quell'applicazione.

Tabella coinvolta: products

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del prodotto
product_key	VARCHAR(100)	Chiave univoca del prodotto, inclusa nella libreria client
name	VARCHAR(255)	Nome leggibile del prodotto
created_at	TIMESTAMP	Data e ora di inserimento del record

2.2 Registrazione del cliente

Quando un cliente installa per la prima volta un'applicazione del produttore, la libreria client avvia automaticamente il processo di registrazione in due fasi: invio dei dati e verifica OTP.

C1 – Registrazione cliente

POST /api/client/register

La libreria client invia i dati anagrafici: `product_key`, P.IVA o VAT number, ragione sociale, paese, email di contatto, lingua preferita e opzionalmente telefono e referente.

Il servizio esegue una **validazione formale** della P.IVA/VAT (formato corretto per il paese indicato) e una **validazione sostanziale tramite VIES**, il sistema UE che verifica che la P.IVA esista ed sia attiva nel registro europeo. Il risultato viene salvato nel campo `vat_verified`. Verifica poi che la coppia `vat_number` + `country` non sia già associata a quella `product_key`: se lo fosse, il tentativo viene bloccato (vedi sezione 2.8).

Se i dati sono validi, il cliente viene salvato nella tabella **clients** con stato pending e viene inviata un'email OTP.

Tabella coinvolta: clients

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del cliente
vat_number	VARCHAR(30)	P.IVA o VAT number — formato libero, validato tramite VIES
country	VARCHAR(2)	Codice paese ISO (es. IT, DE, FR) — usato per validazione VIES
company_name	VARCHAR(255)	Ragione sociale dell'azienda cliente
contact_email	VARCHAR(255)	Email per notifiche e OTP
language	VARCHAR(2)	Lingua preferita — valori: it, en
contact_phone	VARCHAR(30)	Telefono — opzionale

referent_name	VARCHAR(255)	Nome e cognome del referente — opzionale
vat_verified	BOOL	TRUE se la P.IVA è stata verificata con successo tramite VIES
status	ENUM	Stato: pending (attesa OTP) oppure active
created_at	TIMESTAMP	Data e ora di inserimento del record

Tabella coinvolta: otp_codes

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del record
client_id	INT	Riferimento al cliente in clients
code	VARCHAR(10)	Codice OTP generato (es. 6 cifre)
expires_at	TIMESTAMP	Scadenza del codice (es. 15 minuti)
used_at	TIMESTAMP	Data utilizzo — NULL se non ancora usato
created_at	TIMESTAMP	Data e ora di generazione

In questa fase viene inviata la seguente email al cliente:

Email OTP → Cliente (event_code: REGISTRATION_OTP)

Oggetto: Verifica il tuo indirizzo email – {product_name}

Gentile {company_name},

abbiamo ricevuto la sua richiesta di registrazione a {product_name}.

Per completare la registrazione e attivare la Trial Demo, la invitiamo a inserire il seguente codice di verifica nell'applicazione:

Codice OTP: {otp_code}

Il codice è valido per {otp_expiry_minutes} minuti. Se non ha effettuato questa richiesta, può ignorare questa email.

Per maggiori informazioni ci rendiamo disponibili ai seguenti contatti:

Email: {contact_email}

Telefono: {contact_phone}

Cordiali saluti,

Il team di {product_name}

C3 – Nuovo OTP (caso: OTP scaduto o non ricevuto)

POST /api/client/resend-otp

Se il cliente non riceve l'OTP o se scade prima che venga inserito, la libreria può richiederne uno nuovo. Il servizio invalida il codice precedente nella tabella `otp_codes` e ne genera uno nuovo, inviando nuovamente l'email di verifica.

C2 – Verifica OTP

POST /api/client/verify-otp

Il cliente inserisce il codice OTP nell'applicazione e la libreria lo invia al servizio. Se valido e non scaduto, il servizio:

- Attiva il cliente (`status` → `active`)
- Genera la `license_key` univoca con salt random
- Crea la licenza `trial` nella tabella `licenses`
- Genera il file crittografato per uso offline (`license_file`) e la chiave di decrittazione (`license_decrypt_key`) con stessa scadenza della licenza
- Emette JWT + refresh token, incluso il parametro `check_frequency_hours`
- Triggera il GET ALARM verso l'ERP del produttore

Tabella coinvolta: licenses

Campo	Tipo	Descrizione
<code>id</code>	INT	Identificativo univoco della licenza
<code>client_id</code>	INT	Riferimento al cliente in <code>clients</code>
<code>product_id</code>	INT	Riferimento al prodotto in <code>products</code>
<code>license_key</code>	VARCHAR(255)	Chiave univoca generata con salt random — identificativo usato dalla libreria client
<code>license_type</code>	ENUM	Tipo: <code>trial</code> , <code>monthly</code> , <code>annual</code> , <code>provisional</code>
<code>status</code>	ENUM	Stato: <code>active</code> , <code>expired</code> , <code>suspended</code>
<code>max_users</code>	INT	Numero massimo utenti — NULL per <code>trial</code> se non definito
<code>starts_at</code>	TIMESTAMP	Inizio validità
<code>expires_at</code>	TIMESTAMP	Scadenza
<code>activated_at</code>	TIMESTAMP	Data di attivazione
<code>deactivated_at</code>	TIMESTAMP	Data disattivazione — NULL se ancora attiva
<code>vendor_synced</code>	BOOL	FALSE finché il produttore non conferma via F4
<code>license_file</code>	TEXT	File crittografato (Base64) con dati licenza per uso offline — rigenerato ad ogni attivazione/rinnovo
<code>license_decrypt_key</code>	VARCHAR(512)	Chiave di decrittazione — stessa scadenza della licenza, rigenerata ad ogni attivazione/rinnovo
<code>created_at</code>	TIMESTAMP	Data di inserimento del record

Tabella coinvolta: client_tokens

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del record
client_id	INT	Riferimento al cliente in clients
refresh_token	VARCHAR(512)	Refresh token (hash) — ruotato ad ogni utilizzo
expires_at	TIMESTAMP	Scadenza del refresh token (1 ora)
revoked	BOOL	TRUE se revocato anticipatamente
created_at	TIMESTAMP	Data e ora di emissione

In questa fase vengono inviati i seguenti messaggi:

Email di benvenuto → Cliente (event_code: WELCOME)

Oggetto: Benvenuto in {product_name} – Attivazione Trial Demo

Gentile {company_name},

siamo lieti di comunicarle che la Trial Demo di {product_name} è stata attivata con successo.

Il periodo di prova le permetterà di esplorare le funzionalità del prodotto e valutarne i benefici per la sua azienda. La Trial Demo sarà disponibile fino al {expires_at}.

Al termine del periodo di prova, potrà procedere con l'acquisto di una licenza mensile o annuale per continuare ad usufruire del servizio senza interruzioni.

Per maggiori informazioni ci rendiamo disponibili ai seguenti contatti:

Email: {contact_email}

Telefono: {contact_phone}

Cordiali saluti,

Il team di {product_name}

Messaggio in-app → Cliente (event_code: WELCOME)

Titolo: Benvenuto in {product_name}

Testo: La Trial Demo è stata attivata con successo. Ha a disposizione il periodo di prova fino al {expires_at} per esplorare tutte le funzionalità del prodotto.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

Email nuova registrazione → Produttore

Oggetto: Nuova registrazione cliente – {company_name}

Gentile team,

si comunica che un nuovo cliente si è registrato al servizio.

Dettagli registrazione:

Azienda: {company_name}

Prodotto: {product_name}

Tipo licenza attivata: Trial Demo

Scadenza Trial: {expires_at}

Cordiali saluti,

Il sistema di gestione licenze

2.3 Funzionamento ordinario della licenza e modalità offline

Una volta registrato, il client usa la `license_key` e il JWT per tutte le comunicazioni. La frequenza delle chiamate è definita da `check_frequency_hours` restituito in C2. Quando il client è **offline**, utilizza il file crittografato salvato localmente per verificare la validità della licenza senza chiamare il server. Quando torna online, C4 ha sempre la precedenza sul file locale.

C4 – Verifica stato licenza

GET /api/client/license/status

La libreria client chiama questo endpoint periodicamente per verificare lo stato della licenza. Il servizio controlla JWT e `license_key`, aggiorna lo stato se scaduta (active → expired) e restituisce tipo, scadenza, moduli abilitati e massimo utenti.

Tabella coinvolta: modules

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del modulo
name	VARCHAR(100)	Nome identificativo del modulo (es. modulo_contabilita)
description	VARCHAR(255)	Descrizione leggibile — opzionale
created_at	TIMESTAMP	Data di inserimento del record

Tabella coinvolta: license_modules

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del record
license_id	INT	Riferimento alla licenza in licenses
module_id	INT	Riferimento al modulo in modules

C5 – Poll messaggi in-app

GET /api/client/messages

La libreria client fa polling periodico per ricevere messaggi in-app in coda. Il servizio restituisce i messaggi con `delivered_at` NULL, li marca come consegnati, aggiorna `last_seen_at` in `client_activity_logs` e genera i messaggi dai template in `email_templates`.

Tabella coinvolta: messages

Campo	Tipo	Descrizione
<code>id</code>	INT	Identificativo univoco del messaggio
<code>license_id</code>	INT	Riferimento alla licenza — NULL se non legato a una licenza specifica
<code>template_id</code>	INT	Riferimento al template in <code>email_templates</code>
<code>target</code>	ENUM	Destinatario: client oppure vendor
<code>channel</code>	ENUM	Canale: email oppure in_app
<code>type</code>	ENUM	Tipo: banner, alert, info
<code>language</code>	VARCHAR(2)	Lingua: it oppure en
<code>title</code>	VARCHAR(255)	Titolo del messaggio
<code>body</code>	TEXT	Corpo con placeholder già sostituiti
<code>cta_url</code>	VARCHAR(500)	URL call-to-action — opzionale
<code>delivered_at</code>	TIMESTAMP	NULL finché non consegnato
<code>created_at</code>	TIMESTAMP	Data di creazione del messaggio

Tabella coinvolta: email_templates

Campo	Tipo	Descrizione
<code>id</code>	INT	Identificativo univoco del template
<code>event_code</code>	VARCHAR(50)	Codice evento (es. WELCOME, TRIAL_EXPIRING, PROVISIONAL_ACTIVATED, ecc.)
<code>channel</code>	ENUM	Canale: email oppure in_app
<code>target</code>	ENUM	Destinatario: client oppure vendor
<code>language</code>	VARCHAR(2)	Lingua: it oppure en
<code>subject</code>	VARCHAR(255)	Oggetto email — NULL per in_app
<code>title</code>	VARCHAR(255)	Titolo in-app — NULL per email
<code>body</code>	TEXT	Corpo con placeholder (es. {company_name}, {expires_at}, {days_remaining}, ecc.)
<code>created_at</code>	TIMESTAMP	Data inserimento
<code>updated_at</code>	TIMESTAMP	Data ultima modifica

Tabella coinvolta: client_activity_logs

Campo	Tipo	Descrizione
-------	------	-------------

id	INT	Identificativo univoco del record
license_id	INT	Riferimento alla licenza in licenses
last_seen_at	TIMESTAMP	Data e ora dell'ultima chiamata C5 ricevuta
inactivity_notified_at	TIMESTAMP	Data invio email inattività al produttore — NULL se non ancora inviata

C6 – Rinnovo token cliente

POST /api/client/token/refresh

Il JWT del client ha durata molto breve. Quando scade, la libreria usa il refresh token per ottenerne uno nuovo. Il refresh token viene ruotato ad ogni utilizzo. Se anche il refresh token è scaduto (dopo 1 ora di inattività), il client dovrà contattare il produttore per ripristinare l'accesso.

2.4 Sincronizzazione con il produttore

Ogni volta che un cliente completa la registrazione, il servizio invia immediatamente un GET ALARM all'ERP del produttore.

O1 – GET ALARM verso ERP produttore

GET {vendor_erp_url}/alarm

Unica chiamata uscente del server. Viene triggerata da: nuova registrazione, scadenza imminente, attivazione licenza provvisoria, conferma pagamento. I valori di alarm_code possibili sono NEW_REGISTRATION, LICENSE_EXPIRING, LICENSE_EXPIRED, PROVISIONAL_ACTIVATED, PAYMENT_CONFIRMED.

Tabella coinvolta: alarm_logs

Campo	Tipo	Descrizione
id	INT	Identificativo univoco del log
alarm_code	ENUM	Tipo di evento: NEW_REGISTRATION LICENSE_EXPIRING LICENSE_EXPIRED PROVISIONAL_ACTIVATED PAYMENT_CONFIRMED
license_id	INT	Riferimento alla licenza — NULL se non applicabile
sent_at	TIMESTAMP	Data e ora invio
response_status	INT	Codice HTTP restituito dall'ERP
success	BOOL	TRUE se l'ERP ha risposto 200

F3 – Recupero nuove iscrizioni

GET /api/vendor/registrations/new

Ricevuto il GET ALARM, il produttore chiama questo endpoint per scaricare le nuove iscrizioni non ancora processate (vendor_synced = false). Il servizio restituisce tutti i dati del cliente e della licenza, inclusi license_key e vat_verified.

F4 – Conferma ricezione nuove iscrizioni

POST /api/vendor/registrations/confirm

Il produttore conferma di aver ricevuto e processato le iscrizioni. Il servizio marca le relative licenze con `vendor_synced = true`.

2.5 Attivazione licenza a pagamento

Quando un cliente acquista una licenza mensile o annuale, il produttore gestisce il pagamento sul proprio sistema e notifica il servizio tramite F5.

F5 – Attivazione licenza a pagamento

POST /api/vendor/license/activate

Il produttore invia i dettagli della nuova licenza. Il servizio verifica l'esistenza del cliente, controlla la coerenza delle date, disattiva la licenza precedente (inclusa eventuale provvisoria), crea la nuova licenza attiva e **rigenera il file crittografato offline e la chiave di decrittazione**.

In questa fase vengono inviati i seguenti messaggi:

Email attivazione licenza → Cliente (event_code: LICENSE_ACTIVATED)

Oggetto: Licenza {product_name} attivata – Accesso confermato

Gentile {company_name},

siamo lieti di comunicarle che la sua licenza di {product_name} è stata attivata con successo.

Dettagli licenza:

Tipo: {license_type}

Valida fino al: {expires_at}

Può accedere al servizio da subito.

Per maggiori informazioni ci rendiamo disponibili ai seguenti contatti:

Email: {contact_email}

Telefono: {contact_phone}

Cordiali saluti,

Il team di {product_name}

Messaggio in-app → Cliente (event_code: LICENSE_ACTIVATED)

Titolo: Licenza attivata

Testo: La licenza di {product_name} è stata attivata con successo ed è valida fino al {expires_at}. Può accedere a tutte le funzionalità incluse nel suo abbonamento.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

2.6 Avvisi di scadenza e disattivazione automatica

Job schedulati gestiscono le notifiche anticipate di scadenza. Gli avvisi sono **sospesi durante una Licenza Provvisoria attiva**, poiché il cliente ha già dimostrato l'intenzione di rinnovare.

Trial Demo: avvisi 7, 3 e 1 giorno prima della scadenza.

Licenza mensile: avvisi 7, 3 e 1 giorno prima della scadenza.

Licenza annuale: avvisi a 3 mesi, 2 mesi, 6 settimane, 1 mese, 3 settimane, 2 settimane, 10 giorni, 7 giorni, 3 e 1 giorno prima.

Per ogni avviso viene inviata un'email e un messaggio in-app al cliente e triggerato un GET ALARM con alarm_code = LICENSE_EXPIRING.

Messaggi inviati per ogni avviso di scadenza (il testo è lo stesso, cambia solo {days_remaining}):

Email scadenza imminente → Cliente — esempio Trial Demo (event_code: TRIAL_EXPIRING)

Oggetto: La tua Trial Demo di {product_name} scade tra {days_remaining} giorni

Gentile {company_name},

le ricordiamo che il periodo di prova di {product_name} si concluderà tra {days_remaining} giorni, il {expires_at}.

Per continuare ad usufruire del servizio, la invitiamo a procedere con l'acquisto di una licenza.

Al termine della Trial Demo, senza un abbonamento attivo, l'accesso verrà sospeso automaticamente.

Email: {contact_email}

Telefono: {contact_phone}

Cordiali saluti,

Il team di {product_name}

Email scadenza imminente → Cliente — esempio licenza mensile (event_code: MONTHLY_EXPIRING)

Oggetto: La tua licenza mensile di {product_name} scade tra {days_remaining} giorni

Gentile {company_name},

le ricordiamo che la sua licenza mensile scadrà tra {days_remaining} giorni, il {expires_at}.

Per garantire la continuità del servizio, procedere con il rinnovo prima della data di scadenza.

Email: {contact_email}
Telefono: {contact_phone}

Cordiali saluti,
Il team di {product_name}

Email scadenza imminente → Cliente — esempio licenza annuale (event_code: ANNUAL_EXPIRING)

Oggetto: La tua licenza annuale di {product_name} scade tra {days_remaining} giorni

Gentile {company_name},

le ricordiamo che la sua licenza annuale scadrà tra {days_remaining} giorni, il {expires_at}.

Per garantire la continuità del servizio, procedere con il rinnovo prima della data di scadenza.

Email: {contact_email}
Telefono: {contact_phone}

Cordiali saluti,
Il team di {product_name}

Messaggio in-app → Cliente (vale per tutti i tipi di licenza in scadenza)

Titolo: Licenza in scadenza (es. Trial Demo in scadenza / Licenza mensile in scadenza / Licenza annuale in scadenza)

Testo: La licenza di {product_name} scadrà tra {days_remaining} giorni, il {expires_at}. Per garantire la continuità del servizio è necessario procedere con il rinnovo o l'acquisto di un abbonamento.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

Il giorno stesso della scadenza, se la licenza non è stata rinnovata, il servizio la disattiva automaticamente (status → expired) e triggera GET ALARM con alarm_code = LICENSE_EXPIRED.

Messaggi inviati il giorno della scadenza:

Email licenza scaduta → Cliente (event_code: LICENSE_EXPIRED)

Oggetto: Licenza {product_name} scaduta – Accesso sospeso

Gentile {company_name},

le comunichiamo che la sua licenza di {product_name} è scaduta in data {expires_at} e l'accesso al servizio è stato sospeso.

Per ripristinare l'accesso, la invitiamo a contattarci:

Email: {contact_email}
Telefono: {contact_phone}

Cordiali saluti,
Il team di {product_name}

Messaggio in-app → Cliente (event_code: LICENSE_EXPIRED)

Titolo: Licenza scaduta

Testo: La licenza di {product_name} è scaduta il {expires_at}. L'accesso al servizio è stato sospeso. Per ripristinare l'accesso è necessario procedere con il rinnovo o l'acquisto di una nuova licenza.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

Email licenza scaduta → Produttore

Oggetto: Licenza scaduta – {company_name} – {product_name}

Gentile team,

si comunica che la licenza del seguente cliente è scaduta e l'accesso è stato sospeso.

Dettagli:

Azienda: {company_name}
Prodotto: {product_name}
Data scadenza: {expires_at}

Cordiali saluti,
Il sistema di gestione licenze

2.7 Licenza Provvisoria

La Licenza Provvisoria tutela sia il produttore che il cliente nel periodo tra l'emissione della fattura e la ricezione del pagamento. Evita che il cliente usufruisca di una licenza completa senza pagare, garantendo allo stesso tempo la continuità del servizio durante il normale iter amministrativo. Durante la licenza provvisoria tutti gli avvisi di scadenza vengono sospesi.

F7 – Emissione fattura verso cliente

POST /api/vendor/invoice/issued

Il produttore comunica di aver emesso una fattura verso un cliente, specificando la durata della licenza provvisoria (es. 30 giorni). Se la licenza corrente è in scadenza o già scaduta, il servizio attiva automaticamente una Licenza Provvisoria, **rigenera il file crittografato offline e la chiave di decrittazione**, sospende gli avvisi di scadenza e triggera GET ALARM con alarm_code = PROVISIONAL_ACTIVATED.

In questa fase vengono inviati i seguenti messaggi:

Email Licenza Provvisoria → Cliente (event_code: PROVISIONAL_ACTIVATED)

Oggetto: Licenza provvisoria attivata – {product_name}

Gentile {company_name},

in seguito all'emissione della fattura da parte del nostro team, le comunichiamo che è stata attivata una

Licenza Provvisoria per {product_name}, valida fino al {expires_at}.

Questo le garantisce la continuità del servizio nel periodo di elaborazione del pagamento.

Non riceverà ulteriori avvisi di scadenza durante questo periodo.

Per maggiori informazioni ci rendiamo disponibili ai seguenti contatti:

Email: {contact_email}

Telefono: {contact_phone}

Cordiali saluti,

Il team di {product_name}

Messaggio in-app → Cliente (event_code: PROVISIONAL_ACTIVATED)

Titolo: Licenza provvisoria attivata

Testo: È stata attivata una Licenza Provvisoria per {product_name}, valida fino al {expires_at}. Il servizio è garantito durante il periodo di elaborazione del pagamento.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

F8 – Conferma pagamento ricevuto

POST /api/vendor/invoice/paid

Il produttore conferma la ricezione del pagamento. Il servizio disattiva la licenza provvisoria, crea la nuova licenza definitiva, **rigenera il file crittografato offline e la chiave di decrittazione**, riattiva gli avvisi di scadenza e triggera GET ALARM con alarm_code = PAYMENT_CONFIRMED.

In questa fase vengono inviati i seguenti messaggi:

Email attivazione licenza → Cliente (event_code: LICENSE_ACTIVATED)

Oggetto: Licenza {product_name} attivata – Accesso confermato

Gentile {company_name},

siamo lieti di comunicarle che la sua licenza di {product_name} è stata attivata con successo.

Dettagli licenza:

Tipo: {license_type}

Valida fino al: {expires_at}

Può accedere al servizio da subito.

Email: {contact_email}
Telefono: {contact_phone}

Cordiali saluti,
Il team di {product_name}

Messaggio in-app → Cliente (event_code: LICENSE_ACTIVATED)

Titolo: Licenza attivata

Testo: La licenza di {product_name} è stata attivata con successo ed è valida fino al {expires_at}. Può accedere a tutte le funzionalità incluse nel suo abbonamento.

Per maggiori informazioni contattare {contact_email} oppure {contact_phone}.

Caso: pagamento non ricevuto entro la scadenza provvisoria. La licenza viene disattivata automaticamente senza avvisi anticipati. Viene generato solo un messaggio in-app al momento della disattivazione.

In questa fase viene generato solo il seguente messaggio in-app (nessuna email):

Messaggio in-app → Cliente (event_code: PROVISIONAL_EXPIRED)

Titolo: Licenza scaduta

Testo: La Licenza Provvisoria di {product_name} è scaduta il {expires_at}. L'accesso al servizio è stato sospeso. Per ripristinare l'accesso contattare il produttore.

Contattare {contact_email} oppure {contact_phone}.

2.8 Caso eccezionale: tentativo di ri-registrazione bloccato

La license_key viene emessa una sola volta per ogni coppia cliente+prodotto. Se un client tenta di registrarsi nuovamente con la stessa P.IVA e product_key, il servizio blocca il tentativo con errore 409 e genera un messaggio in-app che spiega come procedere.

In questa fase viene generato solo il seguente messaggio in-app (nessuna email):

Messaggio in-app → Cliente (event_code: REGISTRATION_BLOCKED)

Titolo: Registrazione non consentita

Testo: Non è possibile completare la registrazione. La Trial Demo per questo prodotto è già stata utilizzata in precedenza con questo account. Per accedere nuovamente al servizio è necessario contattare direttamente il produttore.

Contattare {contact_email} oppure {contact_phone}.

Questo messaggio viene inserito nella tabella messages con channel = in_app e delivered_at = NULL, in modo che venga restituito alla prossima chiamata C5 del client.

2.9 Monitoraggio attività client e notifica inattività

Ad ogni chiamata C5, il servizio aggiorna `last_seen_at` nella tabella `client_activity_logs`. Un job schedulato verifica periodicamente se ci sono client con licenza attiva che non effettuano chiamate C5 da almeno 7 giorni consecutivi. In tal caso invia un'email al produttore e aggiorna `inactivity_notified_at` per evitare invii duplicati.

In questa fase viene inviata la seguente email al produttore (nessun messaggio in-app):

Email client inattivo → Produttore (event_code: CLIENT_INACTIVE)

Oggetto: Attenzione: client inattivo da 7 giorni – {company_name} – {product_name}

Gentile team,

si comunica che il seguente cliente non risulta attivo sul servizio da almeno 7 giorni. Il sistema non ha ricevuto chiamate dalla libreria client installata presso di loro.

Dettagli:

Azienda: {company_name}

Prodotto: {product_name}

Licenza: {license_type}

Scadenza licenza: {expires_at}

Ultima attività rilevata: {last_seen_at}

Si consiglia di verificare con il cliente se il servizio è ancora in uso o se si sono verificati problemi tecnici.

Cordiali saluti,

Il sistema di gestione licenze

2.10 Riapertura dell'applicazione da parte di un cliente già registrato

Quando un cliente già registrato riapre l'applicazione, la libreria client chiama nuovamente C1. Il servizio riconosce che `vat_number` + `product_key` sono già associati a una licenza attiva e restituisce direttamente i dati della licenza esistente (`license_key`, tipo, scadenza) senza avviare un nuovo processo di registrazione e senza generare messaggi in-app.

La libreria client utilizza poi la `license_key` e il meccanismo di token refresh (C6) per riottenere un JWT valido e riprendere il funzionamento ordinario (C4 e C5).